

GUÍA DE ESTUDIO

Tema 3 · Datos Ausentes y Normalización

Técnicas de Aprendizaje Automático · Maestría IA · UNIR

BLOQUE	SUBTEMAS CLAVE
1. Atributos redundantes y duplicados	Correlación de Pearson · Chi-cuadrado (χ^2) · drop_duplicates()
2. Datos ausentes	MCAR · MAR · NMAR · Eliminación · Imputación estadística · EM · MI
3. Normalización	Min-Max Scaler · Z-score (StandardScaler)
4. De nominal a binario	One-Hot Encoding · get_dummies() · N-1 columnas

Metodología: estudio socrático interactivo · Complemento CS229 Stanford (Andrew Ng)

1. Atributos Redundantes y Duplicados

¿Qué es un atributo redundante?

Un atributo es redundante cuando su información puede derivarse completamente de otro atributo del dataset. Mantenerlo no aporta información nueva al modelo, pero incrementa el costo computacional e introduce multicolinealidad.

1.1 Correlación de Pearson — variables continuas

El coeficiente de Pearson mide la asociación lineal entre dos variables cuantitativas continuas. Rango: [-1, 1].

$$P = \text{Cov}(X,Y) / (S_x \times S_y)$$

Valor de P	Interpretación	¿Redundancia?
$\approx +1$	Correlación positiva perfecta	Sí — eliminar uno
≈ -1	Correlación negativa perfecta	Sí — eliminar uno
≈ 0	Independencia lineal	NO

■ 3 Reglas de Oro (Tema 2):

1. Correlación $\neq$ causalidad
2. $P = 0 \neq$ independencia (solo mide linealidad)
3. Pearson solo detecta relaciones lineales

1.2 Prueba Chi-cuadrado (χ^2) — variables categóricas

Cuando las variables son nominales/categóricas, no aplica Pearson. Se usa la prueba Chi-cuadrado para determinar si dos variables son estadísticamente independientes.

$$\chi^2 = \sum [(Y_{\text{observado}} - Y_{\text{esperado}})^2 / Y_{\text{esperado}}]$$

Paso	Concepto
1. Hipótesis	H0: variables independientes · H1: variables asociadas
2. Nivel α	Típicamente $\alpha = 0.05$
3. Tabla contingencia	Frecuencias observadas por cruce de categorías
4. Frecuencia esperada	$E = (\text{total fila} \times \text{total columna}) / \text{total general}$
5. Calcular χ^2	Suma de $(\text{obs} - \text{esp})^2 / \text{esp}$ para todas las celdas
6. Grados libertad	$df = (\text{filas}-1) \times (\text{columnas}-1)$
7. Valor p	Calculado con χ^2 y grados de libertad
8. Decisión	$p \leq \alpha \rightarrow$ Rechazar H0 (variables asociadas $\rightarrow$ redundancia potencial)

Python — Chi-cuadrado:

```
from scipy.stats import chi2_contingency
stat, p, dof, expected = chi2_contingency(tabla_contingencia)
if p <= 0.05: print('Variables asociadas - posible redundancia')
```

1.3 Registros Duplicados

Los registros duplicados son filas cuyos valores coinciden en todos los atributos seleccionados. Pueden originarse por entrada manual errónea, integración de fuentes o corrupción de datos.

```
df.duplicated() # Detectar duplicados df = df.drop_duplicates() # Eliminar duplicados
```

2. Datos Ausentes

2.1 Mecanismos de pérdida de datos

Un mecanismo de pérdida de datos describe el proceso o razón por la cual un valor está ausente. Comprender el mecanismo es esencial para elegir el método de tratamiento correcto.

Mecanismo	Siglas	Definición	Sesgo
Missing Completely At Random	MCAR	La ausencia no depende de ningún valor observado ni ausente	Ninguno
Missing At Random	MAR	La ausencia depende de otras variables observadas	Manejable
Not Missing At Random	NMAR	La ausencia depende del valor mismo que falta	Grave — requiere modelar la ausencia

2.2 Identificar valores ausentes en Python

```
dataset.isnull().sum() # Conteo de NaN por columna
dataset[[1,2,3]].replace(0, 'nan') # Reemplazar 0 inválidos por NaN
print(dataset.head(20)) # Inspección visual
```

2.3 Técnicas de tratamiento

Técnica	Método Python	Cuándo usar	Riesgo
Eliminación	dropna(inplace=True)	Pocos valores ausentes (<5%)	Pérdida de datos
Imputación por media	strategy='mean'	Variables continuas simétricas	Sensible a outliers
Imputación por mediana	strategy='median'	Variables continuas con outliers	Bajo
Imputación por moda	strategy='most_frequent'	Variables categóricas	Bajo
KNN	KNNImputer()	Datos con estructura de vecindad	Costo computacional
EM (Esperanza-Maximización)	Iterativo	Distribuciones gaussianas	Subestima errores estándar
Imputación Múltiple (MI)	MCMC	Cuando se necesita máxima precisión	Alto costo computacional

Python — SimpleImputer:

```
from sklearn.impute import SimpleImputer
from numpy import nan
imputer = SimpleImputer(missing_values=nan, strategy='mean')
transformed = imputer.fit_transform(dataset.values)
```

3. Normalización y Escalado de Atributos

Los algoritmos que calculan distancias euclidianas (KNN, K-Means, SVM, Redes Neuronales, Regresión Logística, PCA, Descenso por Gradiente) son sensibles a la escala de los atributos. Un atributo con magnitudes grandes dominará injustamente el cálculo de distancias.

3.1 Normalización Min-Max

Escala todos los valores al rango [0, 1] (o [-1, 1]). Preserva la forma de la distribución original.

$$x_norm = (x - x_min) / (x_max - x_min)$$

```
from sklearn.preprocessing import MinMaxScaler scaler = MinMaxScaler() X_norm = scaler.fit_transform(X)
```

3.2 Normalización Z-score (Estandarización)

Transforma los datos a una distribución normal estándar con media = 0 y desviación estándar = 1. El resultado indica cuántas desviaciones estándar se aleja cada valor de la media.

$$z = (x - \mu) / \sigma$$

```
from sklearn.preprocessing import StandardScaler scaler = StandardScaler() X_norm = scaler.fit_transform(X)
```

3.3 Comparativa Min-Max vs Z-score

Criterio	Min-Max	Z-score
Rango de salida	[0, 1] fijo	No acotado (centrado en 0)
Distribución resultante	Conserva la original	Normal estándar ($\mu=0$, $\sigma=1$)
Sensibilidad a outliers	ALTA — distorsiona rango	Más robusta
Cuándo usar	Rango conocido, sin outliers extremos, redes neuronales	Outliers presentes, min/max desconocidos, distribución aprox. normal
Algoritmos que NO necesitan escalado	—	Árboles (CART, Random Forest, Gradient Boosting), Naive Bayes, LDA

■ Problema de outliers en Min-Max:

Si existe un valor extremo (outlier), el denominador ($x_max - x_min$) se infla, comprimiendo todos los demás valores normalizados cerca de 0 y perdiendo precisión digital en la representación. En ese caso se prefiere Z-score.

4. De Nominal a Binario (One-Hot Encoding)

Algoritmos como SVM y Redes Neuronales no pueden procesar variables nominales directamente. Codificarlas como enteros enteros (rojo=1, verde=2, azul=3) introduce un orden jerárquico artificial que no existe → relaciones de desigualdad falsas.

Solución: One-Hot Encoding (Codificación 1-a-N)

Se reemplaza la variable nominal por N columnas binarias (una por categoría). Para cada instancia, solo una columna vale 1 y el resto 0.

Nombre	Sexo	→	Hombre	Mujer
Juan	Hombre		1	0
Angie	Mujer		0	1
Dago	Hombre		1	0

■ ■ Trampa de la Variable Ficticia (Dummy Variable Trap):

Si una variable tiene N categorías, solo se necesitan N-1 columnas binarias. La N-ésima columna es completamente redundante (se puede deducir de las demás), introduciendo multicolinealidad perfecta. Ejemplo: si es_hombre=0, entonces es_mujer=1 automáticamente.

Python:

```
# Con Pandas (más común) df_dummies = pd.get_dummies(df['Sexo']) # genera N columnas # Con
Scikit-Learn from sklearn.preprocessing import OneHotEncoder encoder = OneHotEncoder(drop='first')
# drop='first' evita la dummy trap
```

5. Flujo Completo de Preprocesamiento

Orden recomendado antes de entrenar cualquier modelo de ML:

Paso	Acción	Herramienta Python
1	Detectar y eliminar atributos redundantes (Pearson / χ^2)	<code>df.corr()</code> · <code>chi2_contingency()</code>
2	Detectar y eliminar registros duplicados	<code>df.duplicated()</code> · <code>drop_duplicates()</code>
3	Identificar datos ausentes y elegir estrategia (MCAR/MAR/NMAR)	<code>isnull().sum()</code> · <code>SimpleImputer()</code>
4	Escalar atributos numéricos (Min-Max o Z-score)	<code>MinMaxScaler()</code> · <code>StandardScaler()</code>
5	Codificar variables nominales (One-Hot, N-1 columnas)	<code>get_dummies()</code> · <code>OneHotEncoder()</code>

■ ¿Por qué este orden?

No tiene sentido imputar datos en columnas que luego se eliminarán por redundantes, ni en filas que se eliminarán por duplicadas. El orden minimiza trabajo computacional innecesario y garantiza que las transformaciones se apliquen sobre datos ya limpios.

6. Preguntas Tipo Examen con Respuesta

P1. ¿Cuándo usar Chi-cuadrado en lugar de Pearson para detectar redundancia?

✓ Cuando las variables son categóricas/nominales. Pearson requiere variables cuantitativas continuas.

P2. ¿Qué mecanismo de pérdida de datos es el más grave y por qué?

✓ NMAR (Not Missing At Random), porque el valor ausente depende del valor mismo que falta. No puede estimarse correctamente con los datos observados; es necesario modelar el mecanismo de ausencia.

P3. ¿Por qué la mediana es preferible a la media para imputar cuando hay outliers?

✓ La mediana es un estimador robusto: no se ve afectada por valores extremos. La media incorpora todos los valores en su cálculo, distorsionándose hacia los extremos.

P4. ¿Qué problema tiene Min-Max cuando hay outliers extremos?

✓ El outlier infla el denominador ($x_{\max} - x_{\min}$), comprimiendo todos los demás valores cerca de 0 y perdiendo precisión digital. En ese caso se prefiere Z-score.

P5. ¿Qué algoritmos NO necesitan normalización?

✓ Los basados en árboles: CART, Random Forest, Gradient Boosting. También Naive Bayes y LDA, que manejan internamente las diferencias de escala.

P6. ¿Por qué se usan N-1 columnas en One-Hot Encoding?

✓ Para evitar la trampa de la variable ficticia (dummy variable trap). La N-ésima columna es redundante porque se puede deducir de las demás, introduciendo multicolinealidad perfecta.

P7. ¿Cuál es la diferencia entre normalización y estandarización?

✓ Normalización (Min-Max): escala al rango [0,1], conserva la distribución original. Estandarización (Z-score): transforma a distribución normal estándar ($\mu=0, \sigma=1$), no tiene rango fijo.

P8. ¿Cuándo es inviable eliminar filas con datos ausentes?

✓ Cuando el porcentaje de datos faltantes es alto. En el ejemplo del tema, se eliminaba el 48.9% de los datos, lo cual introduce sesgo por reducción excesiva de la muestra.

7. Resumen de Comandos Python

Función / Método	Descripción
<code>df.corr()</code>	Matriz de correlación de Pearson
<code>chi2_contingency(tabla)</code>	Prueba Chi-cuadrado entre variables categóricas
<code>df.duplicated()</code>	Detecta filas duplicadas (True/False por fila)
<code>df.drop_duplicates()</code>	Elimina filas duplicadas del DataFrame
<code>df.isnull().sum()</code>	Conteo de valores NaN por columna
<code>df.replace(0, 'nan')</code>	Reemplaza 0 inválidos por NaN
<code>dataset.dropna(inplace=True)</code>	Elimina filas con cualquier NaN
<code>SimpleImputer(strategy='mean')</code>	Imputa con media (también 'median', 'most_frequent')
<code>MinMaxScaler().fit_transform(X)</code>	Normalización Min-Max → rango [0,1]
<code>StandardScaler().fit_transform(X)</code>	Estandarización Z-score → $\mu=0$, $\sigma=1$
<code>pd.get_dummies(df['col'])</code>	One-Hot Encoding con Pandas
<code>OneHotEncoder(drop='first')</code>	One-Hot Encoding con Scikit-Learn (evita dummy trap)

Conexión con CS229 Stanford (Andrew Ng)

El preprocesamiento de datos cubierto en este tema es el paso obligatorio previo a cualquier algoritmo de ML del CS229. Específicamente:

- La normalización es esencial para que el Descenso por Gradiente converja eficientemente.
- La detección de redundancia mediante correlación es la base conceptual de PCA.
- La imputación correcta garantiza que las métricas de evaluación del modelo sean válidas.